

Sistema Unespão

Documentação Arquitetural — Modelo C4 e Princípios SOLID

Padaria Unespão

Plataforma digital onde clientes montam lanches personalizados no estilo Subway/Spoletto, via totem físico ou aplicativo móvel.

EQUIPE

David Almeida Saragó · Guilherme Molina Trindade
Lucas Costa Oliveira · Thiago Mitsuo Nomura da Silva

1. Introdução

Este documento apresenta a documentação arquitetural completa do Sistema Unespão, uma plataforma para padarias que permite a montagem e o pedido de lanches personalizados pelos clientes. A documentação está estruturada em três grandes blocos: (1) descrição do projeto e dos conceitos de domínio, (2) modelagem arquitetural seguindo o Modelo C4 nos níveis de Contexto, Containers e Componentes, e (3) discussão de como os cinco princípios SOLID foram aplicados ao backend da solução.

O documento serve como artefato de referência da disciplina de Engenharia de Software II e consolida o conteúdo das iterações realizadas pela equipe ao longo do semestre.

1.1. Finalidade do Sistema

Otimizar o processo de pedido e personalização de produtos, melhorar a experiência do cliente e aumentar a fidelização através de sugestões personalizadas na padaria Unespão.

1.2. Minimundo

O sistema da padaria Unespão gerencia Clientes, que podem realizar pedidos de produtos personalizados. A personalização é feita a partir de Produtos Base e uma variedade de Ingredientes adicionais, permitindo a criação de Itens Personalizados no estilo “Subway/Spoletto”. Cada Pedido registra os itens escolhidos pelo cliente. Após a conclusão do pedido, os clientes podem fornecer uma Avaliação de Prato, que inclui nota e comentários sobre o Item Personalizado. Com base no Histórico de Pedidos e nas avaliações, o sistema gera Sugestões Personalizadas de lanches para clientes recorrentes, visando aumentar a satisfação e a recorrência. A interação do cliente com o sistema ocorre através de Totens/Aplicativos. Para garantir a disponibilidade dos itens, o sistema também considera o Gerenciamento de Estoque de Produtos Base e Ingredientes.

2. Conceitos-chave do Domínio

O minimundo da Unespão estrutura-se em torno de dez conceitos centrais. Cada um define uma entidade ou um mecanismo essencial para a operação da plataforma.

- **Cliente** — entidade que realiza pedidos. Pode ser um cliente novo ou recorrente, com informações básicas e histórico de pedidos associado.
- **Produto Base** — itens fundamentais que servem como ponto de partida para a personalização de um lanche (ex.: tipo de pão, massa de bolo, base de salada).

- **Ingrediente** — opções adicionais e complementares que o cliente escolhe para personalizar seu Produto Base (ex.: recheios, molhos, coberturas, acompanhamentos).
- **Pedido** — transação completa de um cliente, contendo um ou mais Itens Personalizados solicitados.
- **Item Personalizado** — combinação única de um Produto Base com os Ingredientes selecionados, refletindo a customização desejada pelo cliente.
- **Avaliação de Prato** — mecanismo de feedback sobre um Item Personalizado, geralmente com nota (ex.: 1 a 5 estrelas) e campo para comentários.
- **Sugestão Personalizada** — funcionalidade que utiliza o Histórico de Pedidos e as Avaliações de Prato para propor novos Itens Personalizados que o cliente “pode gostar”, aumentando a relevância da oferta.
- **Totem / Aplicativo** — interfaces digitais (totem físico em loja ou aplicativo móvel) através das quais os clientes interagem com o sistema para montar e realizar seus pedidos.
- **Histórico de Pedidos** — registro detalhado de todos os pedidos feitos por um Cliente ao longo do tempo, crucial para análise de preferências e geração de sugestões.
- **Gerenciamento de Estoque** — controle e monitorização dos Produtos Base e Ingredientes disponíveis, garantindo que os itens solicitados possam ser preparados e evitando ruptura de insumos.

3. Visão Arquitetural — Modelo C4

A documentação arquitetural do Unespão adota o Modelo C4, que organiza a comunicação da arquitetura em quatro níveis de zoom progressivo. Cada nível responde a perguntas diferentes para públicos diferentes — de stakeholders de negócio a desenvolvedores.

3.1. Por que escolhemos o C4

O C4 oferece uma linguagem visual simples e padronizada que permite descrever o sistema sem precisar reescrever a arquitetura para cada audiência. Os quatro níveis são:

- **Nível 1 — Contexto:** o sistema e quem interage com ele (usuários e sistemas externos).
- **Nível 2 — Containers:** as grandes peças tecnológicas (aplicações, APIs, bancos de dados).
- **Nível 3 — Componentes:** as partes internas de cada container e suas relações.
- **Nível 4 — Código:** classes, interfaces e detalhes de implementação.

Neste documento detalhamos os três primeiros níveis, suficientes para comunicar a arquitetura ao corpo docente e à equipe técnica que dará continuidade ao projeto.

4. C4 Nível 1 — Diagrama de Contexto

O Diagrama de Contexto apresenta o Sistema Unespão como uma caixa preta, mostrando apenas os atores que interagem com ele e os sistemas externos dos quais depende. É a visão de mais alto nível, voltada para stakeholders de negócio.

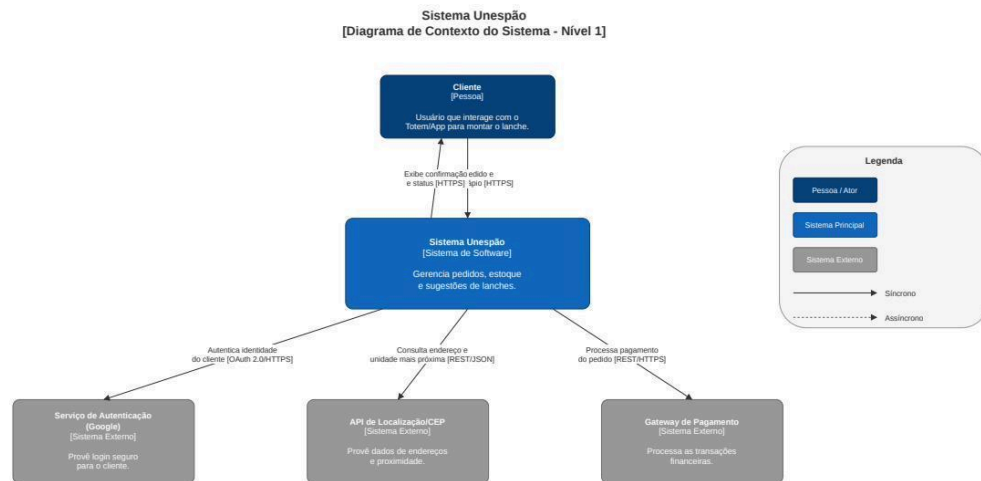


Figura 1 — Diagrama de Contexto do Sistema Unespão (C4 Nível 1).

4.1. Atores e Sistemas Externos

- **Cliente [Pessoa]** — usuário que interage com o Totem ou App para montar o lanche.
- **Sistema Unespão [Sistema de Software]** — núcleo da solução; gerencia pedidos, estoque e sugestões de lanches.
- **Serviço de Autenticação (Google) [Sistema Externo]** — provê login seguro para o cliente via OAuth 2.0.
- **API de Localização/CEP [Sistema Externo]** — fornece dados de endereços e proximidade para identificar a unidade mais próxima.
- **Gateway de Pagamento [Sistema Externo]** — processa as transações financeiras dos pedidos.

4.2. Fluxos de Comunicação

Toda comunicação entre o Cliente e o Sistema Unespão acontece via HTTPS. O Sistema Unespão, por sua vez, conversa com os três sistemas externos: HTTPS para o Google Auth (validação de tokens OAuth 2.0), REST/JSON para a API de Localização/CEP, e REST/HTTPS para o Gateway de Pagamento.

5. C4 Nível 2 — Diagrama de Containers

O Diagrama de Containers abre o Sistema Unespão para mostrar as três grandes peças tecnológicas que o compõem. Esta é a visão voltada para arquitetos e líderes técnicos.

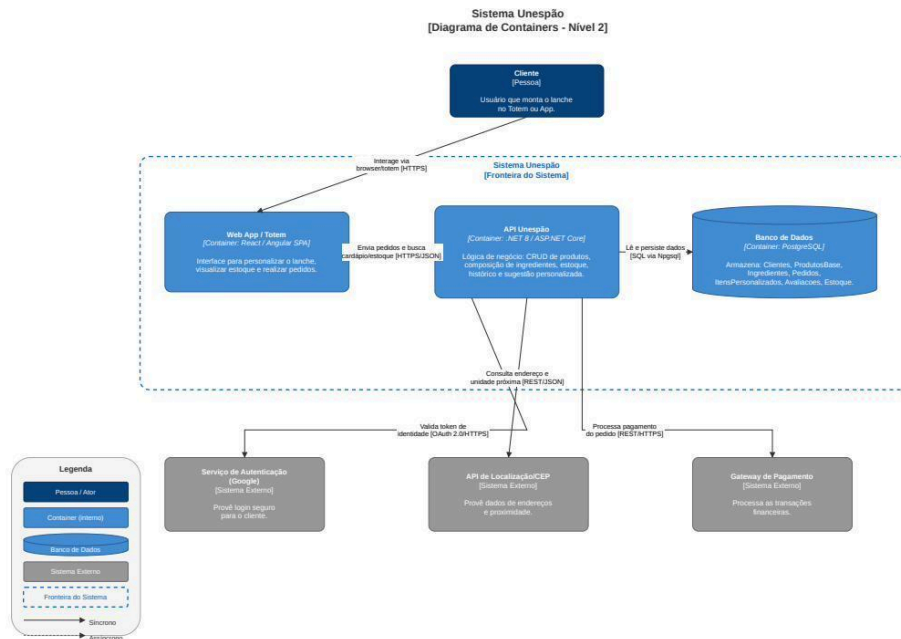


Figura 2 — Diagrama de Containers do Sistema Unespão (C4 Nível 2).

5.1. Containers Internos

- **Web App / Totem [React ou Angular SPA]** — interface para personalizar o lanche, visualizar o cardápio e o estoque, e realizar pedidos. É o ponto de contato direto com o cliente.
- **API Unespão [.NET 8 / ASP.NET Core]** — backend que concentra toda a lógica de negócio: CRUD de produtos, composição de ingredientes, controle de estoque, histórico e sugestão personalizada.
- **Banco de Dados [PostgreSQL]** — armazena Clientes, ProdutosBase, Ingredientes, Pedidos, ItensPersonalizados, Avaliações e Estoque.

5.2. Comunicação entre Containers

O Web App envia pedidos e busca cardápio/estoque pela API Unespão usando HTTPS/JSON. A API persiste e lê dados no banco PostgreSQL via SQL com o provider Npgsql. As integrações com Google Auth, API de Localização/CEP e Gateway de Pagamento partem da API Unespão, nunca diretamente do frontend — o que centraliza segurança e mantém os tokens fora do navegador.

6. C4 Nível 3 — Componentes da API Unespão

O Diagrama de Componentes abre o container API Unespão para revelar sua organização interna. A API segue uma arquitetura em camadas com seis grupos de componentes, com fluxo de dependência unidirecional.

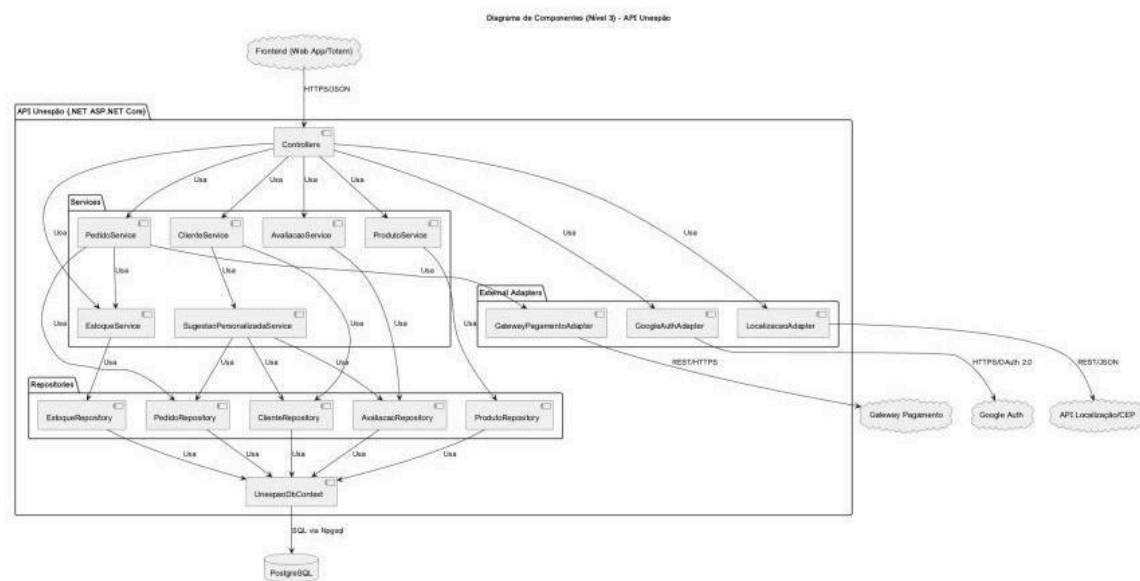


Figura 3 — Diagrama de Componentes da API Unespão (C4 Nível 3).

6.1. Visão Geral dos Componentes

A organização interna reflete uma arquitetura limpa, separando preocupações e facilitando manutenção e escalabilidade. Os componentes estão agrupados em seis camadas:

6.2. Controllers — Porta de Entrada HTTP

Recebem requisições HTTP do Frontend, validam dados de entrada, delegam a execução da lógica de negócios para os serviços apropriados e retornam respostas HTTP em JSON. Não contêm regras de negócio.

Classes principais: ClientesController, PedidosController, ProdutosController, AvaliacaoController e EstoqueController.

6.3. Services — Lógica de Negócios

Concentram o núcleo de inteligência da aplicação. Implementam regras de domínio, coordenam operações entre repositórios e adapters externos, e garantem a consistência dos processos (por exemplo, sincronizar a criação de um pedido com a baixa de estoque).

- **ClienteService** — gerencia clientes, históricos de pedidos e interage com o motor de sugestões.
- **ProdutoService** — gerencia produtos base e ingredientes disponíveis para personalização.
- **PedidoService** — gerencia o ciclo de vida completo dos pedidos, desde a criação até a personalização dos itens e o processamento do pagamento.
- **AvaliacaoService** — processa e armazena avaliações de pratos.
- **EstoqueService** — gerencia o estoque de produtos base e ingredientes, incluindo a baixa de insumos após pedidos.
- **SugestaoPersonalizadaService** — gera sugestões de lanches personalizadas com base no histórico de pedidos e nas avaliações dos clientes.

6.4. Repositories — Persistência

Abstraem os detalhes de persistência de dados, fornecendo uma interface padronizada de operações CRUD para as entidades de domínio. Comunicam-se com o banco através do Entity Framework Core e do provider Npgsql.

Interfaces e classes: `IRepository<T>` (interface genérica), `IClienteRepository`, `IPedidoRepository`, `IProdutoRepository`, `IAvaliacaoRepository` e `IEstoqueRepository`, com suas respectivas implementações concretas.

6.5. External Adapters — Integração Externa

Encapsulam a lógica de comunicação com sistemas externos, traduzindo as chamadas da aplicação para o formato exigido pelas APIs externas e vice-versa. Isolam o sistema de mudanças nesses serviços.

- **GoogleAuthAdapter** — valida tokens de identidade OAuth 2.0.
- **LocalizacaoAdapter** — busca endereços por CEP e identifica unidades próximas.
- **GatewayPagamentoAdapter** — processa pagamentos e consulta status de transações.

6.6. Domain Models — Entidades de Negócio

Representam as entidades de negócio e suas regras de domínio. São objetos ricos que encapsulam dados e comportamento, sem dependência de outras camadas da aplicação.

- **Cliente** — Id, Nome, Email, Telefone, HistoricoPedidos.

- **ProdutoBase** — Id, Nome, Descricao, PrecoBase.
- **Ingrediente** — Id, Nome, PrecoAdicional, Opcional.
- **ItemPersonalizado** — Id, ProdutoBase, IngredientesSelecionados, PrecoTotal, Observacoes.
- **Pedido** — Id, ClientId, DataPedido, Status, Itens, ValorTotal.
- **AvaliacaoPrato** — Id, ItemPersonalizadoId, ClientId, Nota, Comentarios, DataAvaliacao.
- **Estoque** — Id, ProdutoOuIngredienteId, TipoItem, QuantidadeDisponivel, UltimaAtualizacao.

6.7. Infrastructure — DbContext

Gerencia a conexão com o banco de dados e mapeia objetos de domínio para o esquema relacional usando Entity Framework Core.

Classe principal: UnespaoDbContext, contendo as propriedades DbSet<> para Clientes, ProdutosBase, Ingredientes, Pedidos, ItensPersonalizados, AvaliacoesPrato e Estoque.

7. Empacotamento e Fluxo de Dependências

A organização segue os princípios de Clean Architecture, com fluxo unidirecional de dependências. Cada camada conhece apenas a camada imediatamente abaixo.

Fluxo: **Controllers** → **Services** → **Repositories** / **External Adapters** → **Domain Models**. Os Domain Models não dependem de nenhuma camada.

7.1. Benefícios da Organização

- **Testabilidade** — cada camada pode ser testada isoladamente, com mocks substituindo dependências reais.
- **Substituição** — trocar PostgreSQL por outro banco afeta apenas a camada de Repositories.
- **Manutenção** — mudanças em uma camada não vazam para as outras; coesão alta, acoplamento baixo.

8. Princípios SOLID Aplicados

Os cinco princípios SOLID guiaram cada decisão arquitetural do backend Unespão. Os princípios são apresentados na ordem do acrônimo: S, O, L, I, D.

8.1. S — Single Responsibility Principle (SRP)

Princípio. Uma classe deve ter apenas uma razão para mudar.

No Unespão, cada Service tem um único motivo para mudar. Quando uma regra de negócio evolui — por exemplo, uma nova política de pontuação para avaliações — somente uma classe é tocada.

- **PedidoService** cuida exclusivamente do ciclo de vida do pedido.
- **EstoqueService** cuida exclusivamente de estoque e baixa de insumos.
- **AvaliacaoService** cuida exclusivamente de feedback dos pratos.
- **SugestaoPersonalizadaService** cuida exclusivamente de recomendações.

Benefício prático. Os Controllers seguem o mesmo princípio: cada um responde a um conjunto coeso e bem delimitado de endpoints. Mudanças ficam contidas, regressões diminuem.

8.2. O — Open/Closed Principle (OCP)

Princípio. As entidades de software devem estar abertas para extensão, mas fechadas para modificação.

A interface `IRepository<T>` permite acomodar novas entidades sem alterar código existente — basta criar uma nova implementação. O mesmo se aplica aos adapters de pagamento: trocar o gateway de Stripe para PagSeguro, ou adicionar suporte a Pix, significa criar uma nova classe que implementa `IGatewayPagamentoAdapter`. O `PedidoService` não muda nem uma linha.

Resultado. O sistema cresce por extensão (novas implementações), não por alteração de código já testado e em produção.

8.3. L — Liskov Substitution Principle (LSP)

Princípio. Subtipos devem poder substituir seus supertipos sem quebrar o comportamento esperado pelo cliente.

Qualquer implementação de `IPedidoRepository` deve poder substituir outra sem que o `PedidoService` sequer perceba. No projeto Unespão, `PedidoRepository` (PostgreSQL real, usado em produção) e `MockPedidoRepository` (em memória, usado em testes unitários) são intercambiáveis. O mesmo vale para qualquer adapter de pagamento que respeite o contrato esperado.

Atenção. As implementações não podem quebrar invariantes do contrato. Se `IPedidoRepository.GetById` promete retornar um `Pedido` válido, todas as implementações devem honrar essa garantia, sob pena de violar o LSP e produzir falhas silenciosas em produção.

8.4. I — Interface Segregation Principle (ISP)

Princípio. Clientes não devem ser forçados a depender de interfaces que não usam.

Em vez de uma única interface gigante `IService` concentrando toda a lógica, a aplicação tem várias interfaces dedicadas: `IClienteService`, `IPedidoService`, `IAvaliacaoService`, `IEstoqueService` e `ISugestaoPersonalizadaService`. O mesmo padrão se aplica aos repositórios: `IRepository<T>` mantém o CRUD genérico, enquanto interfaces como `IClienteRepository` expõem apenas os métodos específicos do domínio (por exemplo, `GetClienteWithHistorico`).

Por que importa. Cada Controller depende apenas do que realmente usa. O `ClientesController` não é forçado a conhecer métodos de estoque ou pagamento. Acoplamento mínimo, evolução independente.

8.5. D — Dependency Inversion Principle (DIP)

Princípio. Módulos de alto nível não devem depender de módulos de baixo nível. Ambos devem depender de abstrações.

O `PedidoService` não conhece nenhuma classe concreta. Suas três dependências são interfaces: `IPedidoRepository` (e não `PedidoRepository` diretamente), `IEstoqueService` (e não `EstoqueService`) e `IGatewayPagamentoAdapter` (e não `GatewayPagamentoAdapter`).

A injeção de dependência do ASP.NET Core registra as implementações concretas como provedores de cada interface; em runtime, o container DI fornece a instância apropriada para o construtor de cada `Service`. Isso traz três ganhos imediatos:

- **Trocar o banco** afeta apenas os `Repositories`.
- **Mockar dependências em testes** fica trivial — basta passar uma implementação fake da interface.
- **Acoplamento mínimo** entre todas as camadas, permitindo evolução em paralelo.

9. Considerações Finais

A documentação aqui apresentada cobre os três níveis essenciais do Modelo C4 (Contexto, Containers e Componentes) e demonstra a aplicação concreta dos cinco princípios SOLID no backend do Sistema Unespão. A combinação de Clean Architecture com injeção de dependência via ASP.NET Core resulta em um sistema modular, testável e preparado para evoluir junto com o negócio da padaria.

Os artefatos arquiteturais — diagramas C4, organização em camadas e padrões de design adotados — formam uma base coesa para que novos integrantes da equipe possam compreender rapidamente o sistema e contribuir com segurança.

Equipe Unespão

David Almeida Saragó · Guilherme Molina Trindade · Lucas Costa Oliveira · Thiago Mitsuo Nomura da Silva

Engenharia de Software II · 2026